

稠密即更快: 小 Token 规模下稀疏 MoE 的双 GEMM 纯 PyTorch 实现

以 LingBot-VLA 2.0 动作专家为案例的测量、方法与前后对比实验

LingBot-VLA 加速工作组 测试平台:NVIDIA A100-SXM4-80GB 代码分支 `lingbot-v2-accel`(基于 `lerobot PR #3967`)

摘要 流匹配 VLA 策略的去噪循环把稀疏 MoE 专家置于一种极端工况: 每次前向只有 51 个 token, 却要经过 36 层 $\times$ 10 步共 360 次 MoE 调用。本文的测量表明, 此时 MoE 的瓶颈既非 FLOPs 也非显存带宽, 而是 **kernel 发射开销与 torch.compile 图断裂**: A100 上 MoE 的真实 GPU 忙时仅 26.6 ms/次推理, 墙钟却付出 300 ms 以上。据此我们把 top- k 路由 MoE 重写为**两个普通稠密 GEMM**: 将全部 32 个专家的 gate/up 权重按专家维拼接为单个矩阵, 一次 GEMM 算出全部专家的中间激活; top- k 路由权重 one-hot 化后直接折叠进中间激活, 使第二个 (down) GEMM 的归约天然完成“按权重合并专家”——未选中的专家权重恒为 0, 因此与精确路由**代数等价**。该实现纯 PyTorch、静态形状、可微分, 且可被 dynamo 整图追踪。A100 实测: 推理 (与 compile 联用) 1489.1 $\rightarrow$ 398.5 ms (**-73.2%**); 训练 $B=4$ 单步 2807.1 $\rightarrow$ 1501.3 ms (**-46.5%**); 数值差异为浮点重结合级别 (动作相对差 0.78%, 训练 loss 相对差 9×10^{-7})。在非编译回退路径上, 稠密实现 (228.4 ms) 同样快于手写 Triton grouped-GEMM (365.7 ms)。

关键词: Mixture-of-Experts; 视觉-语言-动作模型; 流匹配; kernel 发射开销; torch.compile; 纯 PyTorch 算子

1 引言

LingBot-VLA 2.0 (lerobot PR #3967) 采用 pi0 式双流架构: Qwen3-VL-4B 作为视觉-语言骨干, 并联一个 36 层、hidden 768 的 Qwen2 动作专家; 动作通过流匹配 (flow matching) 生成, 一次推理执行 10 步 Euler 去噪, 每步将 51 个 suffix token (state 1 + action chunk 50) 送过动作专家。动作专家的**全部 36 层**都是稀疏 MoE (32 专家、top-4 路由、专家中间维 512, 另有共享专家 704)。

这意味着一次推理包含 $36 \times 10 = 360$ 次 MoE 前向, 而每次前向仅 51 个 token、204 条路由。MoE 文献中的经典优化目标——大 batch 下的专家并行与显存容量——在此完全错位: 单次调用的计算量只有约 0.48 GFLOP, A100 上纯 GEMM 时间以微秒计。本文要回答的问题是: 在这种“小 T 、高频次”工况下, MoE 的正确实现形态是什么? 答案与直觉相反: 把 32 个专家**全部稠密计算**再按权重丢弃, 比精确路由更快——因为路由机制本身 (argsort、gather、scatter、per-expert 启动) 的成本已经超过了它省下的 FLOPs。

2 动机: 测量先行

对原版实现 (lerobot PR 未修改版, attention 为 eager + fp32) 与中间优化版 (sdpa + bf16 + 循环不变量外提 + 去 sync) 在 A100 上做分段计时与 profiler 核级归因, 得到三个关键事实:

表 1: 去噪路径的结构性测量 (每次推理;GPU 忙时为 profiler 核函数时间求和)

指标	数值	说明
MoE GPU 忙时	26.6 ms	<code>_grouped_gemm_kernel</code> 求和,360 次调用
MoE 墙钟 (非编译)	365.7 ms	同一个 MoE, 发射开销占 93%
整图 GPU 忙时 (compile 后)	225.3 ms	墙钟 ~700 ms, 其余为 launch 间隙
kernel 总数 (compile 后)	22,172	~61 个/层·步, 图被切碎
dynamo graph break	360 / 次推理	每个 MoE 的手写 Triton 内核一个断点

结论清晰: ① MoE 的 GEMM 本身几乎不耗时; ②手写 Triton grouped-GEMM 对 dynamo 是黑盒, 每个 MoE 层强制一次图断裂,inductor 只能融合断点之间的碎片; ③因此墙钟的 ~70% 是 kernel 发射间隙。优化的正确目标不是“更快的 GEMM”, 而是“让整步去噪成为一张完整的计算图”——且只能用纯 PyTorch 算子 (lerobot 社区不接受专用 Triton/GEMM 内核入库)。

3 方法

3.1 记号与常规实现

设 token 数 T , 隐藏维 H , 专家数 E , 专家中间维 I , top- k 路由。输入 $X \in \mathbb{R}^{T \times H}$; 专家 e 的权重 $W_g^{(e)}, W_u^{(e)} \in \mathbb{R}^{I \times H}, W_d^{(e)} \in \mathbb{R}^{H \times I}$ (nn.Linear 约定 [out, in])。路由给出选中专家集合 $\mathcal{S}(t)$ 与权重 $r_{t,e}$ 。常规输出:

$$Y_t = \sum_{e \in \mathcal{S}(t)} r_{t,e} \cdot [\text{SiLU}(x_t W_g^{(e)\top}) \odot (x_t W_u^{(e)\top})] W_d^{(e)\top} \quad (1)$$

分组 (grouped) 实现先按专家排序 gather 出每专家的 token 块, 再逐专家 (或按专家的 grouped kernel) 做三次小 GEMM, 最后 scatter 回原位——这正是发射开销与图断裂的来源。

3.2 权重打包与路由权重折叠

构造两个 E 主序的打包权重 (推理时一次性缓存):

$$W_1 = [W_g; W_u] \text{ 按专家拼接 } \rightarrow \mathbb{R}^{H \times E \cdot 2I}, \quad W_2 = W_d \text{ 按输入维拼接 } \rightarrow \mathbb{R}^{E \cdot I \times H} \quad (2)$$

前向只用两个普通 GEMM:

$$[G \parallel U] = X W_1 \in \mathbb{R}^{T \times E \cdot 2I} \quad (3)$$

$$M = \text{SiLU}(G) \odot U \in \mathbb{R}^{T \times E \cdot I}, \quad \tilde{M} = M \odot R \quad (4)$$

$$Y = \text{reshape}(\tilde{M}, [T, E \cdot I]) \cdot W_2 \in \mathbb{R}^{T \times H} \quad (5)$$

其中 $R \in \{0\} \cup \mathbb{R}^{T \times E}$ 是把 top- k 权重 one-hot 回稠密路由表 ($e \notin \mathcal{S}(t)$ 处恰为 0)。式 (5) 的 GEMM 归约沿 $E \cdot I$ 维求和, 自动完成式 (1) 中的 $\sum_e r_{t,e}(\cdot)$ ——加权合并不需要任何 scatter/index_add, 它就是 down GEMM 本身。

3.3 等价性与数值

未选中的专家 $r_{t,e} = 0$, 其贡献逐项为 0, 故式 (5) $\equiv$ 式 (1), 差异仅来自浮点重结合 (bf16 GEMM 的累加顺序变化)。实测:fp32 下最大相对差 1.4×10^{-6} , bf16 下 4.8×10^{-3} (GPU), 与 attention 后端切换造成的偏差同量级; 端到端动作相对差 0.78%, 训练 seeded loss 相对差 9×10^{-7} (见 §4)。

3.4 复杂度与适用条件

稠密化把专家 FLOPs 放大了 $E/k = 8$ 倍 (每层 $0.48 \rightarrow 3.85$ GFLOP)。代价成立的条件是“发射成本 > 计算成本”。本例中: 路由机制 (排序 + 32 路 gather/scatter + grouped kernel 的网格早退) 约 15–20 次发射 $\times$ 360 次调用, 而 8 倍 FLOPs 折算的 GPU 时间增量仅约 30–70 ms/次推理 (A100, 小 GEMM 效率按 20–40 TFLOPS 估)——且实测中这部分还被 GEMM 固定开销进一步吸收。门控阈值 `moe_dense_max_tokens` (默认 512) 刻画交叉点: $T \leq \text{阈值}$ 走稠密, 否则回退 grouped/eager 路径。注意“排序 + 定容 padding”方案在本工况下退化: 容量上界 $\text{cap} = T = 51, \text{pad}$ 后的形状 $[32, 51, H]$ 与稠密全算完全相同——所以稠密是严格更简且同价的形态。

3.5 与 torch.compile 的协同

两个 matmul + 逐点运算, 全程静态形状、无 `.item()/tolist()`、无自定义内核——`fullgraph=True` 直接通过。去噪步的 36 层得以融合为一张图, 这是 §4 中 compile 收益的主要成分。梯度路径同样成立: autograd 下不读权重缓存 (每次现算打包, 梯度正常回流到三个专家参数), 训练同码复用。

Listing 1: 稠密双 GEMM MoE 核心前向 (推理路径, 缓存打包权重)

```
w1, w2 = self._dense_packed_weights() # [H, E*2I], [E*I, H]
gu = (hidden_states @ w1).view(T, E, 2 * I)
inter = F.silu(gu[..., :I]) * gu[..., I:]
w = torch.zeros(T, E, dtype=torch.float32, device=hidden_states.device)
w.scatter_(1, selected_experts, routing_weights.to(torch.float32))
inter = inter * w.unsqueeze(-1).to(inter.dtype)
return (inter.reshape(T, E * I) @ w2).to(hidden_states.dtype)
```

4 实验

设置。 A100-SXM4-80GB, 真实 6B 权重 (SO101 配置: 1 路相机、6 维状态、chunk 50、10 步去噪), bf16。before = PR 原始代码 (eager+fp32 attention、grouped MoE); dense = 本文实现 (默认阈值 512); compile = predict_velocity 经 inductor 编译 (cudagraphs 关闭)。**注意:** 测量日宿主机有其它任务的 CPU 负载 ($\text{load} \approx 57/72$ 核), 绝对值较空闲机偏高, 但 before/after 同窗口背靠背测量, 相对比较有效; 空闲复测已在队列中。

4.1 推理时延

表 2: 端到端推理 (每次 predict_action_chunk, 出 50 步动作; 20 次迭代均值)

配置	总时 (ms)	Δ vs before
before(PR 原版)	1489.1	—
dense MoE(非编译)	967.4	-35.0%
dense MoE + compile	398.5	-73.2%
参考: 官方 upstream(bf16+compile, 3 相机)	391.2	—

非编译逐环节: dense 路径下 MoE 360 次调用合计 **228.4 ms**, 已快于 Triton grouped 的 365.7 ms (-38%)——两个 GEMM 对十五次以上的杂项发射。compile 后零 recompile 告警, 去噪整步成为单图, 这是 398.5 ms 的主要来源。作为参照, 官方 upstream 仓库 README 最快路径 (厂商 Triton

MoE + 整 sample_actions 编译) 同机实测 391.2 ms: 本文以纯 PyTorch 实现与之打平, 且对方的 compile 范围还包含了本文尚未编译的 vision/prefix 段。

4.2 训练吞吐

表 3: 训练单步 (fwd+bwd, 不含 optimizer)

配置	before (ms)	dense (ms)	Δ	峰值显存 (GB)
$B=4(T=204 \leq 512, \text{走稠密})$	2807.1	1501.3	-46.5%	50.3
$B=16 + \text{grad-ckpt(默认阈值, 回退 eager 循环)}$	8979.5	8979.5	0	28.7
$B=16 + \text{grad-ckpt(阈值 1024, 走稠密)}$	8979.5	6809.9	-24.1%	28.7

训练侧的收益其实更大: 原实现的回退是 32 专家的 Python 循环 (每层 96 次小 GEMM 发射), 稠密化把它压成 2 次。B=16 时 8 倍 FLOPs 开始变成真实成本, 收益收窄但仍为正, 且显存不增。

4.3 数值一致性

表 4: 与 fp32 参考的偏差 (reassociation 级别)

检查	结果
单元级 dense vs eager(fp32, 最大相对差)	1.4×10^{-6}
端到端动作 (dense+sdpa+bf16 vs fp32 参考, 相对差)	0.78%
训练 seeded loss($B=4$, 相对差)	9×10^{-7}

5 讨论

- **何时不该用稠密:** T 很大 (长序列训练、大批次) 时 8 倍 FLOPs 成为真实算力开销, 交叉点由 GEMM 效率与发射成本共同决定, 故保留阈值开关与回退路径。
- **与 CUDA graphs 的关系:** 图完整后 launch 间隙进一步可用 CUDA graphs 消除, 但本次工作按既定约束未使用; 稠密实现与之正交、可叠加。
- **社区可接受性:** 实现不含任何专用内核, 依赖为零增量, 数值与原路径等价, 满足 lerobot 的入库约束。
- **局限:** 表 2/3 测于 CPU 有负载的宿主机, 空闲机绝对值会更低 (before 空闲基线 1039.5 ms); 编译范围目前仅覆盖 predict_velocity, vision/prefix 段的编译是下一步。

6 结论

在“小 T 、高频次”的流匹配 VLA 工况下, 稀疏 MoE 的最优实现不是更精巧的路由调度, 而是把调度整个删掉: 两个打包 GEMM + 权重折叠, 在纯 PyTorch 内同时获得等价数值、整图可编译性与训练推理同码。这提示一类更普遍的判断: 当每层计算微秒级时, 先数 kernel 发射次数, 再谈 FLOPs。

参考与产物

- [1] lerobot PR #3967: LingBot-VLA 2.0 policy(HuggingFace / Robbyant), 改动基线 commit 3b7dbaa0。
- [2] Robbyant/lingbot-vla-v2 upstream 仓库 (官方 deploy 路径与 130 ms @4090D 声明的对照实测)。
- [3] 实现 commit: f2ff2ca5(dense MoE), 测试脚本 bench/bench_lingbot_v2.py, 原始数据 accel-bench/dense_bench_contended/。
- [4] Fedus et al., Switch Transformers (2021);Gale et al., MegaBlocks (2022)——大 T 工况下的分组 MoE 文献, 本文工况与之互补。